



# Radix Sort

The Positional Bucket Distribution Algorithm

**Module 15:** Multi-Pass Architecture

**Series:** CodeHive Pro DSA Suite

# 01. The Radix Principle

---

**Radix Sort** is a non-comparative sorting algorithm that avoids direct "Greater-than" comparisons. Instead, it sorts numbers by processing individual digits. It works based on the **Radix** (base) of the number system (typically base 10 for decimals).

## The "Least Significant Digit" (LSD) Strategy

Radix Sort typically starts from the rightmost digit (ones place), then moves to the tens place, hundreds place, and so on. By sorting one position at a time, the entire array eventually falls into perfect order.

**Key Term: Bucket Sorting.** During each pass, numbers are distributed into 10 "buckets" (0-9) based on the value of the digit currently in focus.

## 02. The Necessity of Stability

---

A critical requirement for Radix Sort is that the underlying sorting mechanism must be **Stable**.

### What is Stable Sorting?

A sort is stable if it preserves the relative order of elements with equal keys. If you have two instances of the number '25', the one that appeared first in the original list must appear first in the buckets.

### Why it matters here:

If the sort were unstable, the work done on the "ones" place would be destroyed during the "tens" place pass. Stability ensures that the order established in previous passes is maintained while sorting the current digit.

## 03. Step-by-Step Execution

---

Initial: [33, 45, 40, 25, 17, 24]

### Pass 1 (Units Place):

- Focus on: 3, 5, 0, 5, 7, 4
- Buckets: [40], [], [], [33], [24], [45, 25], [], [17]
- Rebuilt: [40, 33, 24, 45, 25, 17]

### Pass 2 (Tens Place):

- Focus on: 4, 3, 2, 4, 2, 1
- Buckets: [], [17], [24, 25], [33], [40, 45]
- Rebuilt: [17, 24, 25, 33, 40, 45]

**Final:** The array is sorted across all dimensions.

## 04. Python Implementation

This implementation uses a continuous loop that multiplies the exponent (exp) by 10 in each pass until it exceeds the maximum value.

```
def radixSort(mylist):
    maxVal = max(mylist)
    exp = 1
    radixArray = [[] for _ in range(10)]

    while maxVal // exp > 0:
        # Push into buckets
        while len(mylist) > 0:
            val = mylist.pop(0) # Pop from front to maintain stability
            radixIndex = (val // exp) % 10
            radixArray[radixIndex].append(val)

        # Pull from buckets back into array
        for bucket in radixArray:
            while len(bucket) > 0:
                mylist.append(bucket.pop(0))

        exp *= 10
    return mylist
```

## 05. Big O Performance Analysis

---

The efficiency of Radix Sort depends on the number of elements ( $n$ ) and the number of digits in the largest number ( $k$ ).

$$O(n \cdot k)$$

### Performance Dynamics:

- **Best Case ( $O(n)$ ):** When you have many numbers ( $n$ ) but they all have very few digits ( $k$ ). For example, sorting 1,000,000 integers between 0-999.
- **Constraint:** Like Counting Sort, Radix Sort is limited to **non-negative integers**. It also requires significant extra space to manage the buckets.
- **Worst Case:** When the number of digits  $k$  grows proportionally with  $n$ , making the complexity  $O(n^2)$ .

## 06. Practical Verdict

---

Radix Sort is a powerful alternative to comparison sorts when the data format is predictable (e.g., fixed-width binary or decimal keys).

### Modern Applications:

- **Network Processing:** Sorting IP addresses or packet headers.
- **Data Indexing:** Handling large sets of primary keys in database engines.
- **Multi-Key Sorting:** Useful when sorting elements by year, then month, then day (each can be treated as a "radix").

---

**CODEHIVE ACADEMY**

© 2026. Execution Architect Series. Document ID: SRT-RDX-015.